

Development of a Low-Cost Edge AI Solution for Waste Object Characterization and Mass Estimation Using NVIDIA Jetson Nano and RGB Camera

- Automating waste sorting and mass estimation via real-time, low-cost Edge AI is critical for reducing manual labor costs and improving recycling rates.
- Monocular depth estimation (e.g., Depth Anything) enables metric 3D reconstruction from RGB images, addressing scale ambiguity and cluttered scenes.
- Lightweight object detection models (e.g., YOLOv8-nano, MobileNetV3) balance accuracy and speed, suitable for Jetson Nano's limited compute resources.
- Mass estimation combines density-based methods and regression models, leveraging visual features and depth data to predict weight from 2D/3D cues.
- Optimization techniques including TensorRT quantization, model pruning, and efficient pipeline design achieve real-time performance (≥ 5 FPS) on Jetson Nano.

Executive Summary

The global challenge of waste management demands scalable, cost-effective solutions to automate sorting and mass estimation of waste objects. This report presents a comprehensive, state-of-the-art technical framework for a low-cost Edge AI system leveraging the NVIDIA Jetson Nano paired with an RGB camera to characterize waste objects and estimate their mass in real-time. The proposed solution integrates monocular depth estimation, lightweight object detection and classification, and physics-based or regression-based weight prediction to address the complex, ill-posed problem of mass estimation from 2D images.

Environmental and operational motivations underscore the need for such automation: manual sorting is labor-intensive and costly, while accurate mass estimation enables optimized recycling processes and compliance with waste management regulations. However, key challenges include occlusion, cluttered scenes, scale ambiguity inherent to monocular vision, and the ill-posed nature of inferring mass from visual data alone.

This document synthesizes existing research and proposes optimized architectures, models, and methodologies tailored for edge deployment. It serves as both a theoretical deep dive and a practical implementation guide, detailing dataset considerations, model selection, depth estimation techniques, mass prediction methods, and hardware-specific optimizations. The goal is to balance accuracy, computational efficiency, and hardware affordability, enabling deployment in resource-constrained settings such as recycling facilities, smart bins, or robotic sorting systems.



Background and Related Work

Waste Object Detection and Classification

Waste object detection and classification are foundational tasks for automated sorting. Existing datasets such as TrashNet (2.5K images, 6 classes) and TACO (1.5K high-res images, 60 classes with hierarchical taxonomy) provide annotated waste images but differ in annotation quality, class distribution, and suitability for edge models. TrashNet is limited by its small size and class imbalance, while TACO offers richer annotations and environmental diversity but suffers from overrepresented classes and underrepresentation of dark images, affecting model generalization ^{1 2 3 4}.

Lightweight models optimized for edge devices include YOLOv8-nano, MobileNetV3, and EfficientNet-Lite variants. These models trade off mean average precision (mAP), inference speed, and memory footprint to fit within the constraints of the Jetson Nano's 4GB RAM and limited compute power. For instance, YOLOv8-nano achieves ~6 FPS on Jetson Nano with TensorRT optimization, while MobileNetV3-based models offer better energy efficiency and smaller parameter counts ^{5 6 7 8 9}.

Domain adaptation techniques, such as fine-tuning on combined TrashNet/TACO datasets and synthetic data augmentation, improve generalization to real-world clutter and lighting variations, which are critical for deployment beyond lab environments ^{2 3}.

Monocular Depth Estimation for Metric Reconstruction

Monocular depth estimation is essential to convert 2D images into 3D measurements, enabling volume and mass estimation. Depth Anything, a recent foundation model trained on millions of images, outperforms prior models (e.g., MiDaS, ZoeDepth) in metric depth estimation accuracy and zero-shot generalization. It is well-suited for edge deployment due to its balance between accuracy and latency ^{10 11 12 13}.

MiDaS and other models trained on KITTI and NYUv2 datasets provide robust depth predictions but exhibit biases and artifacts in certain scenarios (e.g., reflections, thin structures). Post-processing techniques like median scaling and CRF refinement improve depth map quality but add computational overhead ^{10 14 15}.

Camera calibration is critical to resolve scale ambiguity and convert depth maps to real-world measurements. Checkerboard-based intrinsic calibration and scale resolution via known object dimensions enable metric reconstruction, which is necessary for accurate volume and mass estimation ^{11 16}.

Mass Estimation from Visual Cues

Mass estimation from visual data is inherently ill-posed due to the lack of direct weight information in images. Density-based methods assume uniform density within object



categories and multiply estimated volume by category-specific densities to predict mass. This approach is simple but sensitive to density variations and shape assumptions ^{17 18 19 20 21}.

Regression-based methods, including linear regression, support vector regression, and neural network-based models, predict mass directly from visual features (e.g., bounding box dimensions, texture, depth statistics) and category embeddings. These methods can capture complex relationships but require high-quality annotated datasets for training ^{22 23 24}.

Hybrid approaches combining density-based priors with regression models show promise in improving accuracy by leveraging both physical principles and data-driven learning. For example, incorporating depth statistics and category-specific embeddings can outperform pure volume-based methods, especially in cluttered scenes ²⁵.

Edge Hardware Benchmarks

The NVIDIA Jetson Nano, with its 128-core Maxwell GPU and 4GB RAM, provides a cost-effective platform for edge AI but has limited compute and power budgets (~5W). Benchmarks show YOLOv8-nano achieves ~6 FPS for object detection with TensorRT FP16 quantization, while larger models like YOLOv8x achieve higher accuracy at the cost of speed and energy ^{8 9 26 27}.

Optimization strategies including INT8 quantization, TensorRT layer fusion, asynchronous pipelines, and model pruning are essential to maximize throughput and minimize latency. These techniques enable real-time inference on Jetson Nano despite its hardware constraints ^{28 29 30}.

Comparisons with other edge devices (e.g., Jetson Orin Nano, Coral TPU) highlight trade-offs in power consumption, performance, and cost. For instance, Jetson Orin Nano offers higher TOPS and power efficiency but at a higher price point, while Coral TPU excels in energy efficiency but has limited framework support ^{31 32 33}.

Methodology

Data Collection and Preprocessing

A custom dataset protocol is proposed to address limitations in existing waste datasets. This protocol specifies:

- **Object Categories:** Align with TACO's 9 super-categories or a simplified taxonomy to balance granularity and annotation feasibility.
- **Annotation Standards:** Bounding boxes, segmentation masks, ground-truth dimensions (via calipers), and weight (via scale). A minimum of 100–200 samples per category is recommended for regression training.
- **Augmentation Strategies:** CutMix, random shadows, and synthetic occlusion to simulate real-world clutter and lighting variations.



Depth calibration follows OpenCV-based checkerboard calibration to estimate camera intrinsics and extrinsics, with error metrics (reprojection error) and tools like Kalibr for stereo setups ^{1 34 11}.

Model Architecture

- **Object Detection/Segmentation:** YOLOv8-n-seg is recommended as the primary model due to its balance of accuracy and speed on Jetson Nano. MobileNetV3-SSD serves as a fallback for faster but less accurate inference. Cascade modes (two-stage detection + classification) can improve accuracy but increase latency ^{5 6 7 8}.
- **Depth Estimation:** Depth Anything (metric mode) is selected for its superior accuracy and robustness. Pre-processing includes resizing to 384×384 for speed/accuracy balance. Camera calibration data is used to convert relative depth to metric scale ^{10 11 12}.
- **Mass Estimation:**
 - **Density-Based:** A table of average densities for common waste materials (e.g., PET plastic, aluminum, cardboard) is used to estimate mass from volume.
 - **Regression-Based:** A lightweight MLP or 1D CNN takes object features (bounding box area, convex hull area, depth statistics), category embeddings, and optional texture features (e.g., LBP histograms) as input. Huber loss is used for robustness to outliers. Evaluation metrics include RMSE, MAPE, and R^2 ^{22 23 24 17 18}.

Dimension Estimation

Algorithmic steps convert depth maps and masks into 3D bounding boxes or convex hulls. Edge cases such as thin objects or reflective surfaces are handled via user-in-the-loop calibration, allowing manual correction of scale via reference objects ^{11 16}.

Real-Time Pipeline

The software stack uses Python + PyTorch/TensorRT + OpenCV. Latency targets are allocated per module (e.g., 50ms detection, 30ms depth, 20ms mass estimation) to achieve ≥ 5 FPS on Jetson Nano. Fallback modes switch to 2D area × category-specific height priors if depth estimation fails ^{26 27 30}.

Experiments

Depth Accuracy

- **Protocol:** Compare Depth Anything (relative/metric) against ground-truth caliper measurements on the custom dataset. Report relative error (%) per object category.
- **Ablations:** Test impact of camera resolution (640×480 vs. 1280×720) and calibration quality (checkerboard vs. self-supervised scaling).
- **Baseline:** Stereo camera (e.g., Intel RealSense D435) for comparison if data is available ^{10 11 12}.



Mass Estimation

- **Baselines:** Density-based method with fixed densities vs. regression model trained on synthetic data.
- **Ablations:** Compare models using 2D features (area) vs. 3D (volume) vs. combined features; evaluate super-categories vs. sub-categories.
- **Error Analysis:** Plot MAPE vs. object size/weight to identify systematic biases [22](#) [23](#) [24](#).

End-to-End Performance

- **Latency:** Measure frame-to-frame time for the full pipeline, broken down by module using nvprof or Jetson's built-in tools.
- **Power:** Report watts and mWh/request under load, comparing Jetson Nano (5W mode) vs. Orin Nano.
- **Robustness:** Test on occluded/cluttered scenes (e.g., objects overlapping by 30–50%) and report detection/mass estimation degradation [26](#) [27](#) [30](#).

Discussion

Limitations

- Monocular depth estimation suffers from scale drift in dynamic scenes and requires calibration.
- Regression models may generalize poorly to unseen categories (e.g., electronics waste).
- Jetson Nano's thermal throttling under sustained load limits continuous operation without cooling.
- Occlusion and clutter remain challenging for both detection and mass estimation.

Alternative Hardware

- Jetson Orin Nano offers better performance and power efficiency but at higher cost.
- Luxonis OAK-D (stereo + AI) provides integrated depth sensing and AI inference at low power.
- Raspberry Pi + Coral TPU is a lower-cost alternative but with limited AI framework support.

Deployment Challenges

- Lighting variations require IR or RGB-IR fusion for robust operation.
- Occlusion handling may need multi-view fusion or temporal tracking.
- Scalability considerations favor edge-cloud hybrid approaches for heavyweight models.



Repository Structure

A detailed README template is provided, including:

- **Installation:** Dockerfile/Conda env for Jetson Nano (CUDA/TensorRT versions).
- **Data Prep:** Scripts to download/convert TrashNet/TACO and generate custom dataset splits.
- **Training:** Configs for YOLOv8, Depth Anything, and regression models (hyperparameters, augmentation params).
- **Inference:** CLI commands to run the pipeline on a webcam stream, with flags for visualization/logging.
- **Evaluation:** Scripts to reproduce all tables/figures from the document.

Future Work

1. **Active Learning:** Use uncertainty estimation to flag low-confidence predictions for human labeling.
2. **Semi-Supervised Depth:** Fine-tune Depth Anything on unlabeled waste images via pseudo-labels.
3. **Robotics Integration:** ROS nodes for sorting arms, with grasp point estimation from depth maps.
4. **Energy Optimization:** Dynamic model switching based on battery level.

Figures and Tables

- **Model Comparison Table:** Accuracy, speed, memory footprint for YOLOv8 variants, MobileNetV3, EfficientNet.
- **Depth Estimation Benchmark Table:** RMSE, $\delta < 1.25$, inference time on Jetson Nano for Depth Anything, MiDaS, RT-MonoDepth.
- **Mass Estimation Error Table:** RMSE, MAPE, R^2 for density-based, regression-based, and hybrid methods.
- **Hardware Benchmark Table:** Latency, power consumption for Jetson Nano vs. Orin Nano vs. Coral TPU.

This comprehensive report synthesizes the state-of-the-art in edge AI for waste object characterization and mass estimation, providing both theoretical insights and practical implementation guidance tailored for NVIDIA Jetson Nano and RGB cameras. The proposed solution balances accuracy, computational efficiency, and cost-effectiveness, addressing critical challenges in waste management automation.

[1] [TACO Dataset](#)

[2] [Object detection and classification for taco litter dataset using faster R-CNN](#) | ScholarWorks



- [3] [Exploring the TACO Dataset \[Data Analysis\] | Encord](#)
- [4] [GitHub - garythung/trashnet: Dataset of images of trash; Torch-based CNN for garbage image classification](#)
- [5] [A comprehensive survey of deep learning-based lightweight object detection models for edge devices | Artificial Intelligence Review | Springer Nature Link](#)
- [6] [\(PDF\) Lightweight and real-time object detection model on edge devices with model quantization.](#)
- [7] [\(PDF\) A comprehensive survey of deep learning-based lightweight object detection models for edge devices](#)
- [8] [Benchmarking Deep Learning Models for Object Detection on Edge Computing Devices](#)
- [9] [YOLOv8 Performance Benchmarks on NVIDIA Jetson Devices - Latest News from Seeed Studio](#)
- [10] [Best Depth Estimation Models: Depth Anything V2 & More](#)
- [11] [Survey on Monocular Metric Depth Estimation](#)
- [12] [Depth Anything](#)
- [13] [GitHub - LiheYoung/Depth-Anything: \[CVPR 2024\] Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data. Foundation Model for Monocular Depth Estimation](#)
- [14] [Depth Pro Explained: Sharp, Fast Monocular Metric Depth Estimation](#)
- [15] [Monocular Depth Estimation with Depth Anything V2 | Towards Data Science](#)
- [16] [Real-time Monocular Depth Estimation on Embedded Systems](#)
- [17] [Solid waste density: Volume-to-weight conversions | Minnesota Pollution Control Agency](#)
- [18] [Characterizing municipal solid waste component densities for use in landfill air space estimates - Carson Cline, Malak Anshassi, Steven Laux, Timothy G Townsend, 2020](#)
- [19] [Investigating the bulk density of construction waste: A big data-driven approach - ScienceDirect](#)
- [20] [Density of Garbage, household rubbish in 285 units of density](#)
- [21] [Physical Characteristics of Solid Waste - Densite, Moisture, Size](#)
- [22] [Municipal Solid Waste Management: A Review of Machine Learning Applications](#)
- [23] [Advanced predictive modeling of municipal solid waste management using robust machine learning | Scientific Reports](#)
- [24] [\(PDF\) Density-based weighting for imbalanced regression](#)
- [25] [Edge Enhanced Scene Understanding and Depth Estimation | Proceedings of the 2025 7th International Conference on Image, Video and Signal Processing](#)
- [26] [Jetson Nano | NVIDIA Developer](#)
- [27] [Jetson Nano Brings AI Computing to Everyone | NVIDIA Technical Blog](#)
- [28] [Get Started With Jetson Nano Developer Kit | NVIDIA Developer](#)
- [29] [Benchmarking Deep Learning Models on NVIDIA Jetson Nano for Real-Time Systems: An Empirical Investigation](#)
- [30] [Deploy YOLOv8 with TensorRT | Seeed Studio Wiki](#)
- [31] [Unlocking the Power of NVIDIA Jetson Nano Developer Kit for AI and Robotics - fibermall.com](#)
- [32] [Which Jetson is best for AI application build? - WizzDev](#)
- [33] [Jetson Nano vs Raspberry Pi 5 for AI: The Ultimate Performance and Val - ThinkRobotics.com](#)
- [34] [TACO - Dataset Ninja](#)

